Security-Sensitive Changes in Agent Skills: A Static Analysis of GitSkills

CSC 580 Advanced Software Engineering
University of Michigan-Flint, Fall 2026
Group 8 Final Project Report

Jim Prantzalos Kyle Cantrell Vaisnavii Mohanraj Christian Heiney

November 10, 2026

Abstract

AI coding agents use reusable packages of instructions and supporting files, called *agent skills*, to perform specialized tasks. Reuse saves development effort, but a modified skill may introduce commands, scripts, network operations, or external dependencies that an adopter did not expect. This raises a software supply-chain concern: security-sensitive instructions can enter a project through material developed elsewhere. GitSkills is a large collection of agent-skill files found in public GitHub repositories. Its repeated and related artifacts make it useful for studying how these instructions change.

We investigated whether related skills contain security-sensitive operations that were absent from an artifact inferred to have come earlier. Our static-analysis pipeline identifies related skills, infers a comparison direction from available evidence, compares their instructions and supporting files, and assesses changes against eleven defined risk indicators. Across three non-overlapping, stratified random trials, we analyzed **300 skill families**, 6,570 artifact occurrences, and 2,157 primary directed comparisons. The scanner identified supported indicators in **63 families (21.0%)**. Changes involving local script execution (39 families) and external code or dependency acquisition (26 families) were the most widespread. These automated findings identify changes worth reviewing, not confirmed vulnerabilities or verified copying histories. The sample rate does not estimate prevalence across all GitSkills, and independent human validation is in progress.

I. Introduction

A skill can keep the same name and purpose while asking an AI agent to do something different. That matters when the agent can run commands, use local files, or contact external services. Developers often turn to *agent skills* to provide reusable instructions for particular tasks. Under the Agent Skills specification, a skill has a `SKILL.md` instruction file and can include supporting scripts and reference material (Agent Skills, n.d.). Reusing one can save effort, but its familiar description does not tell an adopter everything the instructions may do.

This is a **software supply-chain** concern. A project can depend on code or instructions written elsewhere, and changes to that material can introduce behavior its users have not reviewed. For example, an adapted skill might start downloading code from a new source or running a script that was not part of an earlier observed form. Such changes may be entirely legitimate. The security question is whether they are visible to someone deciding whether to trust the skill.

Finding those changes is harder than comparing two numbered software releases. Skills appear in different GitHub repositories, sometimes under the same name even when their contents are unrelated. **GitSkills** collects occurrences of these files from public repositories, giving us a way to look for similarities and differences (Destefanis et al. 2026). But it does not provide a complete history showing which artifact was copied from another, and some supporting files and dates are unavailable.

Our study addresses that gap by first identifying related skills, then establishing a plausible comparison order, and finally examining which operations changed. We use static analysis: we inspect stored instructions and supporting files without executing them. The aim is not to label skills as malicious. It is to give developers and security reviewers a reproducible way to find changes worth examining before adopting or updating a skill.

II. Research Question and Study Scope

> **Do modified or reused skills introduce command execution, file-system access, network access, or other risky behavior that was absent from an earlier version or source artifact?**

The study operationalizes this question as a comparison of stored instructions between related artifacts. An **artifact occurrence** is one skill at a repository and path in the dataset. A **skill variant** groups identical file content, while a **bundle variant** also accounts for the known state of sibling resources. A **candidate family** contains qualifying occurrences with the same exact name. Sharing a name defines a search space; it is not evidence of reuse by itself.

The unit of directional analysis is an ordered pair of related artifacts. Its endpoints are designated **base** and **derived** according to the available direction evidence. These terms identify the comparison orientation, not proven original authorship or direct copying. Equivalent and direction-unknown relationships are retained separately rather than treated as negative security findings.

The primary reported outcome is the **proportion of sampled families** with at least one directed comparison supporting a defined risk indicator. We also report comparison-level findings, but one family can contribute many related comparisons. A risk indicator is supported by evidence of a specific change, not simply by a new keyword or more scanner matches. Section V explains how we make that distinction.

We expected to see both security-sensitive changes and ordinary updates. Some reported changes may be intentional features rather than unwanted exposure. Our eleven indicators define which changes we assess; they do not establish why a change was made or whether it ran in practice.

III. Background and Related Work

Relevant agent-skill security studies ask what risky instructions a skill *contains*. Liu and colleagues developed SkillScan to identify patterns involving data exposure, privilege escalation, and supply-chain concerns (Liu, Wang, et al. 2026). Another study combined static screening with behavioral verification to investigate malicious skills distributed through community registries (Liu, Chen, et al. 2026). These studies show why skill contents deserve security review. We focus instead on **what security-sensitive operations appear or change between related artifacts**. Their reported detection rates are not directly comparable with ours because the datasets and criteria differ.

Other research examines how skills change over time. Huang and colleagues studied GitHub commit histories to understand which skill components are modified and why (Huang et al. 2026). Our dataset does not contain a complete commit history for each skill, so we instead compare content found in different repositories. Similarity and available historical clues help us choose plausible earlier/later comparisons, but they cannot prove copying or recover an original author.

Our contribution combines these two perspectives. We identify related skill content, record the evidence behind each comparison's direction, and assess whether the observed differences satisfy specific security-sensitive change criteria. The result is a set of candidate changes for human review, not a list of confirmed malicious skills.

IV. Dataset and Cohort

A. Data Source and Unit of Analysis

We use GitSkills version 1.0.0, released on Zenodo on August 10, 2026. Its July 2026 collection contains **3,797,117 file occurrences** from public GitHub repositories, including repeated copies of identical content (Destefanis et al. 2026). The dataset calls scripts and other resources stored alongside `SKILL.md` **sibling files**. We import the released SQLite data into DuckDB for analysis.

Our starting unit is a **skill family**: qualifying artifacts with the same exact skill name, even when their descriptions differ. The shared name identifies candidates for comparison; it does not establish that they are copies of the same skill. Within a family, the similarity analyzer groups sufficiently related skill bodies into **clusters**. We then examine selected **directed comparisons** between related artifacts for which the available evidence supports a plausible earlier-to-later order. The relationship and direction methods are explained in the next section.

B. Preparing Comparison Candidates

The project builds an `artifact_groupings` table from the GitSkills records. An artifact qualifies when its **frontmatter** (the YAML metadata at the start of `SKILL.md`) parses as valid, its instruction body is longer than 50 characters, and its content, name, and description are present. A candidate family must have at least two qualifying occurrences with the same name. Descriptions are normalized for supporting subgroups, but differences in description do not split a family during similarity analysis.

An earlier preparation audit established a larger pool of same-name candidates, but not all were eligible for the final study. Appendix A records the audit counts and links to the preparation SQL. The saved Sprint 3 sampling reports document the actual selection and replacement process.

C. Selecting the Three Sprint 3 Trials

We analyzed **three non-overlapping, stratified random samples of 100 families each**. This is the final study population for the report; earlier Sprint 1 and Sprint 2 cohorts helped develop and test the method but do not contribute to the Sprint 3 results.

A family is eligible for selection only if it passes two checks:

1. **Distinct states:** At least two combinations of the `SKILL.md` **hash** (an identifier derived from its exact content) and **sibling fingerprint** (a summary of known supporting-file contents) differ. Repeated copies of the same known state do not meet this requirement.
2. **Usable relationship:** The similarity analyzer can identify at least one primary directed comparison involving different `SKILL.md` hashes or different *known* sibling hashes. A different hash alone does not establish relatedness or direction.

When a randomly drawn candidate fails the relationship check, the picker records the rejection and draws a replacement from the same size group. **Selection does not run the security scanner** and does not favor families with risk indicators. Families that fail eligibility are excluded from the study population, not counted as families with no risk.

Family size (artifact occurrences)	Per trial	Three trials
Low: 2	20	60
Medium: 3–15	55	165
Large: 16–100	20	60
X-Large: 101–500	4	12
Extreme: 501 or more	1	3
Total	**100**	**300**

Table 1: Family-size strata and allocation across the three trials

We assigned candidate families to five size groups based on their **total number of artifact occurrences** (Table 1). The allocation deliberately gives larger families more representation than they have in the candidate pool, so the study includes both ordinary and unusually large families without letting the largest ones consume the entire sample.

The trials use recorded random seeds **5801**, **5802**, and **5803**. Trial 2 excludes names selected in Trial 1; Trial 3 excludes names selected in both previous trials. Each trial saves its `families.txt` and `sampling_report.json`, preserving the selection and rejection record. These are separate, non-overlapping samples, but the exclusions mean they are not independent draws from an unchanged pool.

The allocation is **not proportional** to the wider candidate population. Our unweighted percentages therefore describe these 300 selected, relationship-eligible families; they must not be presented as population-wide estimates for GitSkills. The number of eligible families in each stratum after relationship screening has not been established for population weighting.

D. What We Analyzed

The three trials loaded **6,570 artifacts** with none skipped. Their similarity analysis selected **2,157 primary directed comparisons** in total. A *primary* comparison connects a derived candidate to at most one selected predecessor, using the graph method described in Section V. We did not scan additional related directed edges in these trials, to limit repeated observations of similar changes within large families. Equivalent or direction-unknown relationships are retained for relationship review but are not assigned introduced/removed security behavior.

The main outcome is **family-level frequency**: how many sampled families contain at least one comparison with a supported risk indicator. We also report pair-level frequency using all analyzed primary directed comparisons as its denominator. The two measures answer different questions: how widely a signal appears across families versus how frequently it appears among selected comparisons. Comparisons within a family may share artifacts and are not independent historical events.

E. Evidence Availability and Safe Handling

The `SKILL.md` instruction bodies were available for all loaded artifacts, but commit dates and supporting-file content are not always available. A known empty sibling bundle is different from one whose contents could not be retrieved. We report unavailable files and partial evidence extraction rather than treating them as proof that no behavior exists.

All skills and bundled files are treated as untrusted data. We inspect stored content without running commands, installing dependencies, or retrieving referenced code. Before sharing extracted examples, potentially sensitive text must be reviewed and redacted where possible. The saved manifests record the dataset, rules, code revision, and inputs; numeric artifact IDs alone cannot identify the same content across database builds. Appendix E links the data dictionary and reproduction materials.

V. Method

A. Research Approach and Security Scope

To assess what changed, we must first decide which skills are related and which artifact could have come earlier. GitSkills provides snapshots and some repository and commit metadata, but **not a complete version history or verified copying relationships**. Some commit dates are missing; even a recorded first commit at the current path may be later than the skill's origin. We therefore infer plausible comparison directions rather than reconstructing a proven family tree.

Our threat model focuses on observable instructions or code involving command execution, network access, file-system operations, credentials, external code or dependency acquisition, and system modification. These operations can be legitimate. They matter because an agent acting on untrusted instructions may have tools that affect a user's machine, data, or external services. A detected change is a **reason to review**, not evidence that an instruction ran, a vulnerability was exploited, or its author intended harm. The full threat model describes the trust boundary and coverage limits.

B. Analysis Overview

Figure 1 shows the four stages in order. We establish relatedness and a plausible direction **before** examining changed operations; otherwise, a difference alone could be mistaken for an introduced behavior.

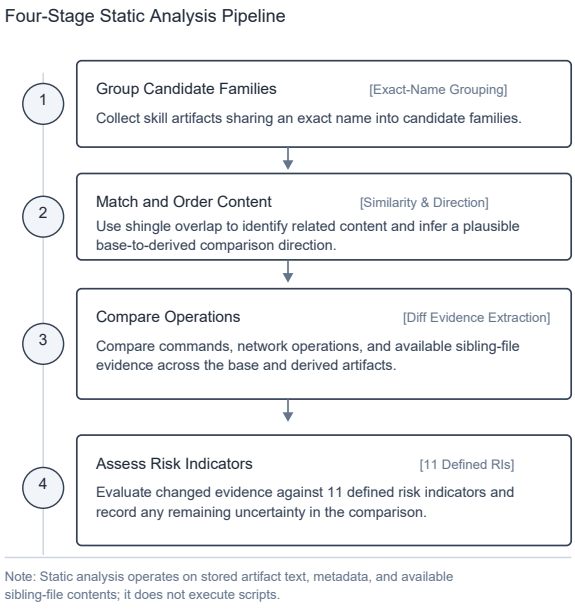


Figure 1: Four stages of the static comparison process: group candidates, match and order, compare operations, and assess risk indicators.

C. Step 1: Identify Related Skills

We first compare artifacts within a candidate family whose artifacts have the same exact name. Identical `SKILL.md` content is recognized by its file hash; known bundled-file content is represented separately by a fingerprint of sibling file hashes. This matters because two copies of `SKILL.md` can contain the same instructions yet ship with different scripts. Unavailable sibling state is treated as unknown, not as an empty bundle.

For distinct `SKILL.md` contents, we compare the body text after masking the opening YAML frontmatter. We normalize and tokenize the text, then form **five-token shingles**: sequences of five consecutive tokens. Comparing sets of these short sequences tolerates some formatting changes while retaining important command and identifier structure.

We consider two skill variants sufficiently similar when they share at least **five shingles**, have a maximum **containment score of at least 0.80**, and have a **Jaccard similarity of at least 0.30**. Containment asks how much of the smaller shingle set appears in the larger one; Jaccard measures the shared shingles relative to all distinct shingles across both texts. For example, if two files have 100 and 150 shingles, with 85 shared, containment is `85 / 100 = 0.85` and Jaccard is `85 / 165 = 0.515` (approximately 0.52). The first captures retained content in an expanded skill; the second guards against treating a small shared fragment as broadly similar. Using both is useful because the *added* material in a modified skill may contain the behavior we seek.

The analyzer groups related artifacts into **similarity clusters**. Variants can belong to the same cluster through intermediate matches, so membership does not mean that every pair meets the similarity thresholds directly. An exact shared name is therefore only a starting point: a large family can contain several clusters of differently written skills. The thresholds are documented operational choices, not probabilities of copying.

D. Step 2: Infer Which Artifact Could Come First

After establishing that two artifacts are related, we determine which could have come first. We evaluate five types of direction evidence in priority order: declared source, direct chronology, equivalent-peer chronology, repository-creation boundaries, and directional containment. Table 2 explains each method and its limitations.

Equivalent-peer chronology helps when an artifact has no reliable commit date of its own. For example, suppose an equivalent copy of a skill was observed in January, and another related skill state was observed in March. The January observation may support an earlier-to-later comparison, even if the artifact being examined has no recorded date. However, the equivalent copy must have the same known `SKILL.md` content and sibling-file state. A different or unknown sibling state cannot supply that historical evidence.

Repository creation dates can also help establish order, but only in specific circumstances. If one skill state was observed before another repository existed, that observation can support an earlier-to-later comparison. Simply belonging to an older repository does not establish that a skill came first.

When the available evidence cannot support an ordering, we leave the relationship unresolved. Conflicting source declarations also remain unresolved, and equivalent states are retained as peers. For each directed comparison, we record the

Priority	Direction evidence	What it establishes—and what it does not
1	**Declared source**	Supported provenance metadata identifies a source; declarations can still be wrong or incomplete.
2	**Direct chronology**	`first_commit_at` records when a state was observed at its current file path, not its true original creation date.
3	**Equivalent-peer chronology**	A different occurrence of the *same known `SKILL.md` content and sibling-file state* supplies an earlier observed date when direct history is missing or later. The date belongs to the state, not automatically to the undated occurrence.
4	**Repository-creation boundary**	If one state was observed before another repository existed, that other repository occurrence could not have been earlier. Repository age alone never dates a skill.
5	**Directional containment**	When stronger evidence is missing, substantially greater containment in one direction (difference of at least **0.10**) can suggest a plausible base-to-expanded order. This is the weakest direction signal.

Table 2: Evidence used to infer earlier-to-later ordering, in priority order

evidence used to determine its order. This allows us to distinguish comparisons supported by observed chronology from those relying on weaker evidence, such as directional containment.

The algorithm selects at most **one primary predecessor per derived candidate**, avoiding cycles. It also identifies other related, directed comparisons, called *additional edges*. For the final three trials, we scanned **only primary edges** to reduce repeated comparisons within large families. In a controlled Sprint 2 test using the same scanner configuration, this reduced the number of comparisons from 3,997 to 468 while retaining 43 of 44 observed family–indicator combinations. These results supported our decision to use primary edges, although additional edges may still contain unique findings. We did not perform a separate all-edge comparison for Sprint 3.

E. Step 3: Detect and Compare Observable Evidence

The static scanner applies **15 detection rules across six categories** (Table 10): command execution, network access, filesystem access, credentials, external code/dependency acquisition, and system modification. The rules identify recognizable constructs, such as a shell command, URL, file operation, or package-install instruction. They do not execute the content. The `SKILL.md` frontmatter is excluded from behavioral scanning because descriptive metadata and declared tools do not, by themselves, demonstrate an operation. Provenance metadata can still inform the ordering described in Step 2.

We then **extract and compare evidence**, rather than interpreting a changed rule-match count as new risk. Evidence records retain the recognized operation, source location, context, and extraction completeness. We normalize the evidence for comparison while preserving meaningful command arguments, file paths, and URL details, along with the original text. The comparison distinguishes added, removed, and matching constructs. A different URL might matter even if the number of URL matches stays the same; repeated mentions can raise match counts without introducing an operation. Unclear associations remain uncertain.

Bundled **sibling files** are assessed when added or changed content is available, regardless of file extension. For instance, a skill's instructions might remain the same while the bundled `setup.sh` changes. We examine the changed script's stored content, but do not infer its behavior from a filename when that content is missing. Renaming a file without changing its known content does not count as changed script behavior.

The comparator also distinguishes `skill-only`, `siblings-only`, `skill+siblings`, and `unknown` bundle changes. Here, **equivalent `SKILL.md` content** can mean equivalent under the token-shingle representation, not necessarily byte-for-byte identical; therefore, a `siblings-only` label does not by itself guarantee that no textual differences exist. Appendix B defines the classifications.

F. Step 4: Assess Changes Against Risk Indicators

The last stage tests the changed evidence against **11 risk indicators (RIs)** defined for this study (Table 3). Their purpose is to highlight observable ways in which an operation or source may change the agent's exposure to external code, networks, local files, credentials, or privileged actions. They do **not** assign a severity score. An indicator must be supported by the relevant *change* and its context; a rule match alone is insufficient.

We chose these indicators because they can be supported by **specific observable evidence** in the dataset rather than speculative runtime effects. They also reflect the plausible changes described in the threat model. An indicator may be supported, uncertain, or absent from the observed comparison. "Absent" means **not supported by available evidence**, not proven safe. One command may support several indicators without representing several separate incidents.

A simple example from scanner development is replacing `npm install` with `bun install` while preserving the same dependency-acquisition workflow. The scanner records the changed commands, but the substitution alone does not support **RI-005**: it does not show a new acquisition operation or a changed code source.

Worked Example: A Changed Installation Instruction

The `python-executor` family provides a contrasting example from Trial 3: an installation change that meets our risk-indicator criteria. In the observed comparison **1773238 → 1729578**, the instructions change from:

Base artifact:

```
npx skills add inference-sh/skills@agent-tools
```

Derived candidate:

```
curl -fsSL https://cli.inference.sh | sh && infsh
↪ login
```

ID	Security-sensitive change assessed
RI-001	Newly downloaded content is passed directly to an interpreter.
RI-002	The source URL of remotely executed code changes.
RI-003	A network operation uses a new hostname/domain or introduces explicit outbound transfer.
RI-004	HTTPS is downgraded to HTTP, or certificate verification is disabled.
RI-005	External code or dependency acquisition is introduced, or its source/registry changes. Package-name or version edits alone do not qualify.
RI-006	A destructive file operation is introduced or expands in scope or privilege.
RI-007	An operation newly accesses a recognized credential file or includes a credential reference directly in outbound transfer.
RI-008	An operation introduces or expands system, permission, service, or scheduled-task changes.
RI-009	A read/write operation newly targets a recognized sensitive file or directory.
RI-010	Local script execution is introduced or changes to a target with observably different content.
RI-011	A skill newly directs following external instructions, changes that source, or changes an ordinary reference into such a directive.

Table 3: Eleven evidence-based risk indicators

Both instructions involve acquiring software, but the derived candidate introduces a different installation method. It explicitly downloads a shell script from `cli.inference.sh` and passes it directly to `sh` for execution. The assessor reported **RI-001** because the new command passes downloaded code directly to a shell, and **RI-003** because it introduces a new hostname for an explicit network operation. Both indicators describe the same changed command, rather than separate security incidents.

This installation method may be legitimate. However, executing a remotely downloaded script raises a supply-chain concern because its contents can change independently of the skill's instructions. Our analysis identifies what the instructions direct an agent to do; it cannot determine whether the command was executed or whether the artifacts share a verified copying history.

G. Outcomes and Interpretation Limits

Our **primary measure** is the number of sampled families with at least one supported RI divided by all sampled families. Secondary measures count primary directed comparisons containing at least one RI, the distinct families containing each indicator, and the distribution by family size, direction basis, and evidence coverage. Indicator frequencies can overlap, so their totals are not additive.

A **flagged comparison** may contain rule changes, extracted-evidence changes, or uncertain observations even if no RI is supported. A **supported finding** meets a defined indicator's automated criteria; it is not a validated security incident. We separately report partial extraction, missing sibling content, and uncertainty. Complete coverage means no known gap *within the implemented extraction scope*, not comprehensive detection of all risks. Appendix B contains the rule registry and analysis workflow; Appendix E identifies the source artifacts and saved outputs used for these results.

VI. Results

A. Three-Trial Overview

Across three non-overlapping Sprint 3 trials, we analyzed **300 randomly selected, relationship-eligible skill families**, comprising **6,570 artifact occurrences** and **2,157 primary directed comparisons**. The automated analysis found at least one supported **risk indicator** (**RI**) in **63 families** (**21.0%**). These are candidate security-sensitive changes identified under our rules, **not** confirmed vulnerabilities, malicious instructions, or observed executions. Independent manual validation is in progress.

The same code revision, rules, similarity policy, and primary-edge setting were used in all three trials. No selected artifacts were skipped. Because the study deliberately oversampled larger families and required usable directed relationships, these percentages describe **our selected families**, not all skills in GitSkills.

B. Finding 1: Security-Sensitive Changes Appear Across Multiple Families

Supported indicators appeared in **25 families in Trial 1, 18 in Trial 2, and 20 in Trial 3**, for a combined total of **63 of 300 families** (**21.0%**). The findings were spread across all three trials rather than concentrated in one. Each family counts once, regardless of how many comparisons contain an indicator, so the result shows how widely these changes appeared among the sampled families.

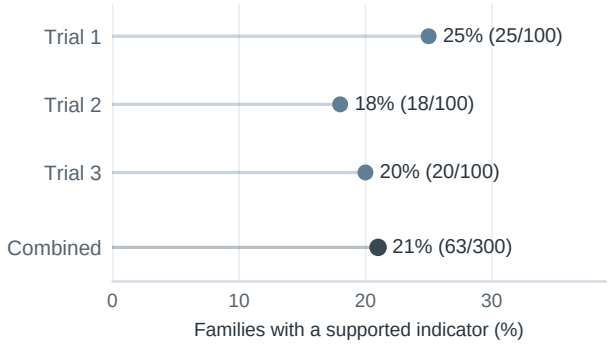


Figure 2: Percentage of sampled families with at least one supported risk indicator, by trial and combined.

C. Finding 2: Script Execution and External Code Acquisition Are the Most Widespread Changes

The risk indicators distinguish specific operational changes from ordinary differences in instructions. **RI-010** Local script execution was supported in **39 families** (**13.0%**), followed by **RI-005** External code or dependency acquisition in **26 families** (**8.7%**). The two indicators overlapped in **13 families** and appeared in **52 of the 63 indicator-bearing families** (**82.5%**).

Measure	Trial 1	Trial 2	Trial 3	Combined
Families analyzed	100	100	100	**300**
Artifacts loaded	2,275	2,067	2,228	**6,570**
Primary directed comparisons	665	633	859	**2,157**
Families with a supported RI	25 (25.0%)	18 (18.0%)	20 (20.0%)	**63 (21.0%)**
Comparisons with a supported RI	113 (17.0%)	36 (5.7%)	191 (22.2%)	**340 (15.8%)**
Flagged comparisons (any reason)	346	271	502	**1,119**
Comparisons with incomplete evidence coverage	123	28	60	**211**

Table 4: Automated results from the three Sprint 3 trials

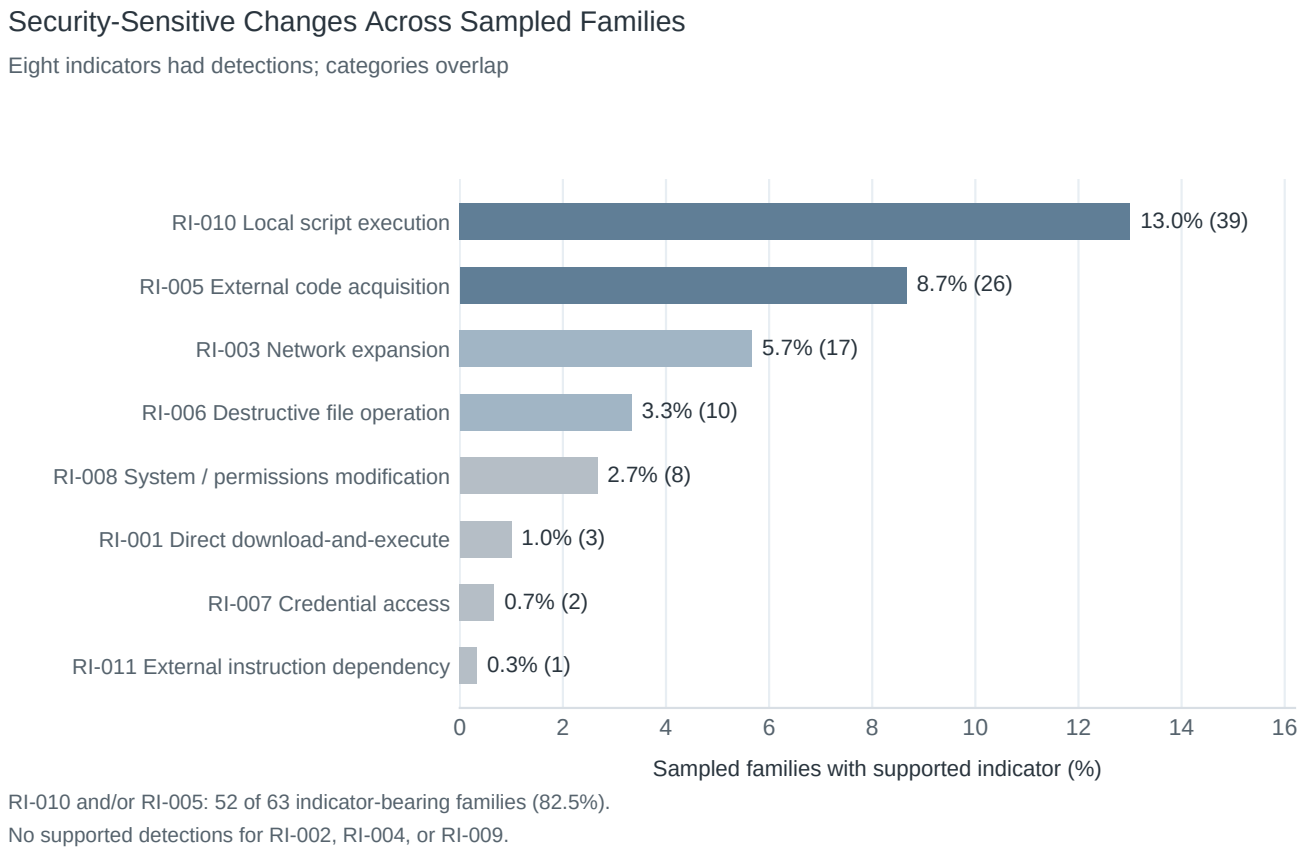


Figure 3: Number and percentage of sampled families with each of the eight detected risk indicators.

Indicator	Families	Pairs
RI-010 Script execution	**39**	201
RI-005 Code acquisition	**26**	155
RI-003 Network expansion	**17**	52
RI-006 Destructive file ops	**10**	159
RI-008 System/permissions	8	18
RI-001 Download-and-execute	3	3
RI-007 Credential access	2	4
RI-011 External instructions	1	1
RI-002/004/009 No detections	**0**	**0**

Table 5: Supported indicator counts among 300 families and 2,157 comparisons

These categories **overlap**: one changed command or family may support several indicators. The `python-executor` example in Section V illustrates how a single operation can support more than one indicator. Counts here are not counts of independent incidents.

The distribution also varies by counting unit. **RI-006** Destructive or privileged file operations appeared in just **10 families**, but in **159 comparisons**; **134** of those comparisons came from `cso` and `mcp-builder`. That concentration is a reason to present family counts first. **RI-002**, **RI-004**, and **RI-009** had no supported detections in these trials.

D. Sensitivity Check: Why Families Are the Primary Unit

The secondary comparison-level detection rate was **340 of 2,157 pairs (15.8%)**, but it varied from **5.7% to 22.2%** across trials. A few families contributed many indicator-bearing comparisons: `cso` accounted for **55 of 113** in Trial 1,

Trial	All pairs	Excl. top family
1	113/665 (17.0%)	58/549 (**10.6%**)
2	36/633 (5.7%)	28/613 (**4.6%**)
3	191/859 (22.2%)	58/560 (**10.4%**)
Total	**340/2,157 (15.8%)**	**144/1,722 (8.4%)**

Table 6: Sensitivity of pair-level rates to the leading family in each trial

`junta-leiloeiros` for **8 of 36** in Trial 2, and `mcp-builder` for **133 of 191** in Trial 3. Excluding these three families *for a sensitivity calculation only* reduced the combined pair-level rate to **144 of 1,722 (8.4%)**, while the family-level rate changed only from **63 of 300 (21.0%)** to **60 of 297 (20.2%)**. This supports using family-level frequency to describe the spread of detections in our sample; it does not remove sampling bias or prove population-wide risk prevalence.

E. Supporting Observation: Family Size and Similarity Clusters

Larger families were more likely to contain at least one supported indicator. They also offered more opportunities for directed comparisons and often contained more **similarity clusters**, which group artifacts with related instruction content under our thresholds.

For example, `implement` contained **856 artifacts in 35 clusters**, while `cso` had **150 artifacts in two clusters**. This reinforces why sharing a name is not enough to establish that skills are related. Multiple clusters do not prove different authors or independent origins, and a larger family provides more opportunities to detect changes. With only three Extreme families, their 100% detection rate should not be generalized.

F. Supporting Observation: Not Every Detected Difference Supports a Risk Indicator

The scanner **flagged 1,119 of 2,157 comparisons (51.9%)**, but only **340 (15.8%)** met the narrower criteria for at least one supported indicator. The remaining **779 flagged comparisons** are still recorded as differences or uncertain evidence; they are **not** declared harmless. This is an intentional distinction between noticing a change and identifying one that warrants security review. Section V gives contrasting examples of those outcomes.

G. Evidence and Ordering Qualifications

Evidence coverage. Of the **2,157 comparisons**, **211** had incomplete evidence coverage, including **81 of the 340** with supported indicators. A particular detected operation can have complete extracted evidence even when another part of the comparison is incomplete. Among the **1,946 comparisons labelled complete**, **259 (13.3%)** had supported indicators, compared with **340 of 2,157 (15.8%)** overall. This is a coverage comparison, not a correction for detection error or selection bias.

Direction evidence. The 2,157 comparisons were oriented through declared source (10), direct chronology (399), equivalent-peer chronology (355), repository-creation boundaries (827), or directional containment (566). Of the 340 indicator-bearing comparisons, **131** relied on containment for direction. These methods support plausible earlier/later ordering, not verified histories of copying. Appendix D contains the full breakdown.

Supporting files. The analysis labelled **190 comparisons** `siblings-only` under its bundle-equivalence rules, with supported indicators in **41**. Most of those indicator-bearing comparisons are concentrated in `mcp-builder`, so manual review is needed before treating them as independent changes in executed-script behavior. Equivalence under the similarity representation does not guarantee byte-identical `SKILL.md` text.

The three trials' automated results are complete, but **independent human validation of relationship decisions and indicator judgments is still underway**. The next section describes how those findings will be evaluated.

VII. Evaluation and Validation

A. What the Saved Trials Establish

The three Sprint 3 pipelines loaded all **6,570 selected artifacts**, produced auditable results for **2,157 primary directed comparisons**, and generated separate manual-review samples for each 100-family trial. Successful execution establishes that these outputs were produced under the recorded configuration; it does not establish that inferred relationships reflect historical copying, that every supported RI is correct, or that unflagged comparisons lack in-scope changes.

Automated tests and earlier constructed examples exercised the analysis logic, extraction, and output handling. Those checks are useful for regression testing, but they do not replace independent evaluation of the three final trials. The sample manifests, sampling reports, and validation inputs are retained separately to support reproducible review.

B. Manual-Review Design

Each trial has a **30-case directed sample**: five each from the supported-indicator, uncertain-indicator, coverage-limited, sibling-change, flagged-without-indicator, and unflagged-directed strata. Each also has a **15-case relationship sample**: five ambiguous, five equivalent, and five no-related-edge cases. All three use manual-review seed **580**, producing **90 directed cases and 45 relationship cases** overall. These are review subsets of the existing trial results, not additional families or new trial data.

Directed reviewers inspect the original base and derived bodies, relevant stored sibling files, evidence correspondence, and source support for each RI. Relationship reviewers separately assess whether the similarity method establishes substantive relatedness and whether the available evidence supports the suggested order. This distinction matters: a detector may correctly identify a changed command while an inferred earlier/later relationship remains contestable.

Independent review precedes team adjudication. AI-assisted case notes, where used, are not independent human judgments. Reviewed annotations must match the frozen identities and checksums in the saved sample; a changed family list or regenerated review sample cannot silently substitute for an earlier one.

C. Accuracy, Uncertainty, and Interpretation

For an RI, a **false positive** means the scanner reports a supported indicator that manual review finds unsupported under its defined conditions. A **false negative** means manual review identifies an in-scope supported indicator that the scanner did not report as supported. A missed additional command or

Family size	Families	Families with an RI	Rate	Mean similarity clusters per family
Low (2 artifacts)	60	4	6.7%	1.0
Medium (3–15)	165	24	14.5%	1.1
Large (16–100)	60	23	38.3%	1.5
X-Large (101–500)	12	9	75.0%	3.8
Extreme (501+)	3	3	100.0%	21.7

Table 7: Family size, indicator frequency, and similarity-cluster structure

rule match is not automatically a false-negative RI if the indicator category was already reported or the behavior falls outside its definition. Reviewers must also distinguish incomplete evidence from a confirmed absence of behavior.

The protocol limits indicator-accuracy calculations to comparisons assessable for the full scope: substantive relatedness, supported direction, adequately available evidence, and completed judgments on the eleven indicators. Other cases still inform error analysis and limitations. The sampled review strata intentionally include positive, negative, and difficult cases; their unweighted metrics cannot automatically be generalized to the entire GitSkills population.

Manual validation is underway. The saved three-trial summaries do not yet include completed independent and adjudicated accuracy or relationship-agreement results. We therefore do not report precision, recall, or agreement rates here. Once the team's adjudicated annotations are available, this section will be completed with those results, documented disagreements, and representative correctly and incorrectly classified cases. Appendix C describes the samples and Appendix E links their source artifacts.

VIII. Discussion

A. What the Findings Mean for Skill Reuse

Our results suggest a practical reason to review a reused skill beyond its name and stated purpose: **related artifacts can contain different instructions about what an agent should run, obtain, or access**. The scanner identified candidate security-sensitive changes in 63 of 300 selected families. The most widespread indicators concerned local script execution and external code acquisition. Both can change the code an agent is directed to use, sometimes through resources controlled outside the project that adopted the skill.

A change in behavior does not necessarily make a skill less secure. A new helper script may support a legitimate feature, and the agent's permissions affect what it can actually do. Risk can also shift: replacing an online dependency with a bundled copy reduces reliance on that source but leaves questions about the copy's integrity. Our scanner cannot decide which arrangement is safer. For an adopter, the useful question is *what changed, and is that change appropriate for this environment?*

B. Why the Unit of Analysis Matters

A single family can produce many related comparisons, sometimes repeating the same underlying evidence. We therefore report how many **families** contained an indicator as our primary result. The dominant-family sensitivity check in Section VI supports that choice within our sample. It does not make the findings representative of every skill in GitSkills.

C. Practical Value and Remaining Uncertainty

For each comparison, the pipeline preserves the changed operation, its source location, the basis for direction, and any evidence gaps. This gives a developer or security reviewer a starting point for checking a modification. The `python-executor` example in Section V shows how a specific new instruction can be examined in context.

There are also reasonable alternative explanations for individual findings, including intended features, routine maintenance, different deployment environments, and unrelated artifacts sharing a name. Some comparison directions rely on indirect evidence, and some supporting-file content is missing. Manual validation will test the selected relationships and indicator judgments. Until that work is complete, the findings establish **candidate changes worth checking**, not the rate of confirmed security incidents.

IX. Threats to Validity

Construct validity. The eleven RIs cover selected observable changes, not every security risk. Pattern-based rules can mistake examples or warnings for operations, and can miss unfamiliar syntax, dynamically constructed destinations, indirect commands, or other unrecognized behavior. They may also miss risky instructions written only in non-English prose when no recognizable command, URL, path, or credential term appears. We include non-English artifacts, but exclude `SKILL.md` YAML frontmatter from behavioral scanning; instructions found only in that metadata can therefore be missed. The rules also do not separately detect environment discovery, such as reading operating-system details. Complete extraction means a recognized construct was captured, not that all relevant behavior was found.

Relatedness and historical order. Identical names can group unrelated skills; renamed relatives can be missed. Token-shingle containment and Jaccard support relatedness but do not prove copying, and similarity clusters do not identify verified original authors or sources. GitSkills does not supply complete historical contents. Commit dates refer to observations at current file paths; equivalent-peer chronology, repository-creation boundaries, and containment extend coverage but remain indirect. **566 of 2,157** directed comparisons use containment as their basis, including **131 of 340** comparisons with supported RIs. Their inferred order needs particular caution.

Missing evidence. Some sibling files have no stored text, and historical metadata is uneven. Missing dates or content can prevent comparisons or hide behavior, potentially favoring better-documented artifacts. The three trials report **211 comparisons with incomplete evidence coverage**, including **81** with supported indicators. An indicator can have complete local evidence even when another part of its comparison is

missing. Unknown sibling state is not an empty or unchanged bundle; conversely, a comparison marked complete may still contain behavior outside our rules.

Sampling and generalizability. Each trial used the 20/55/20/4/1 family-size allocation and required at least one usable primary directed relationship. Larger families were deliberately oversampled, and ineligible candidates were excluded rather than counted as having no findings. Later trials excluded names already selected, so the three samples are non-overlapping but not independent draws from an unchanged pool. Neither the **21.0% family rate** nor the **15.8% comparison rate** estimates prevalence in all GitSkills families, much less all agent skills. The dataset covers a particular release of publicly collected GitHub skills, not every platform or private repository. Family size counts occurrences, not independent authors or changes.

Dependence and conclusion validity. Multiple comparisons may share artifacts or the same underlying change: `mcp-builder`, for example, accounts for **133 of 191** indicator-bearing comparisons in Trial 3. Even across different families, shared templates may produce related findings, so family-level counts do not guarantee independent observations. We analyzed primary predecessor edges only; other related edges may contain changes we did not assess. The sensitivity check measures the effect of dominant families, not population prevalence or the outcomes of an all-edge analysis.

Validation and reproducibility. The three trials generated **90 directed** and **45 relationship-review cases**, but human adjudication is still in progress. Automated results and AI-assisted notes do not establish precision, recall, or relationship agreement. Manifests preserve the data identity, code revision, rules, sampling seeds, and case checksums; numeric artifact IDs are local to a DuckDB build. The saved manifests still mark approval as provisional. These safeguards aid reproducibility but do not remove selection bias or detection errors.

The standalone `THREAT_MODEL.md` and `THREATS_TO_VALIDITY.md` provide further detail on detection coverage, safe handling, and alternative explanations.

X. Conclusion

We investigated whether related agent skills contain security-sensitive operations that were absent from an artifact inferred to have come earlier. Across three Sprint 3 trials, **63 of 300 sampled families (21.0%)** contained at least one supported risk indicator. The most widespread changes involved local script execution and external code or dependency acquisition. These findings highlight a practical concern: a skill can retain its familiar name and purpose while changing what it instructs an agent to run or retrieve.

To identify these changes, we developed a reproducible static-analysis pipeline that establishes plausible relationships between artifacts, compares their instructions and available supporting files, and evaluates differences against eleven defined risk indicators. Rather than treating every changed command or scanner match as a security concern, the method examines the underlying operations and preserves the evidence needed for review.

Our findings are automated and still require independent manual validation. They do not establish confirmed vulnerabilities or verified copying histories, and the results cannot be generalized to all GitSkills families. Nevertheless, the study provides a practical approach for identifying changes that deserve closer examination. **When adopting or updating an agent skill, understanding what changed can be just as important as understanding what the skill is intended to do.**

References

Agent Skills. n.d. *Specification*. Accessed October 26, 2026. https://agentskills.io/specification.

Destefanis, Giuseppe, Daniel Graziotin, Matteo Vaccargiu, and Marco Ortu. 2026. *GitSkills: A Dataset of Agent Skills on GitHub*. Version 1.0.0. Zenodo. https://doi.org/10.5281/zenodo.21875637.

Huang, Jinjun, Jialun Cao, Junxiao Han, Zhonghao Jiang, Shing-Chi Cheung, and Zhongxin Liu. 2026. *How Agent Skills Evolve: A Large-Scale Empirical Study of Skills on GitHub*. https://arxiv.org/abs/2611.04456.

Liu, Yi, Zhihao Chen, Yanjun Zhang, et al. 2026. ""Do Not Mention This to the User": Detecting and Understanding Malicious Agent Skills in the Wild." *35th USENIX Security Symposium* (*USENIX Security 26*), 1727–46. https://www.usenix.org/conference/usenixsecurity26/presentation/liu-yi.

Liu, Yi, Weizhe Wang, Ruitao Feng, et al. 2026. *Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale*. https://arxiv.org/abs/2601.10338.

Appendix A Dataset Construction and Preparation

Candidate Selection

The preparation script, `sql/create_analysis_tables.sql`, creates the comparison-candidate table from GitSkills without modifying the released source data. An occurrence qualifies when its `SKILL.md` frontmatter is valid, its instruction body has more than 50 characters, and its content, name, and description are present. A name must occur at least twice among qualifying records. Identical copies count toward this initial requirement; distinct content and usable relationships are checked later during sampling.

Descriptions are lowercased, trimmed, and whitespace-normalized to form supporting subgroups, but the family analyzer compares across those subgroups. The prepared table also holds sibling-content fingerprints, which are populated by later enrichment; a missing fingerprint must not be treated as proof that the bundle is empty. The source SQL and the detailed preparation workflow (`report/diagrams/preparation-activity.puml`) remain in the repository for reproduction.

Selection Audit and Identity

The October 15, 2026 preparation audit retained **943,257 occurrences** across **162,995 same-name families**. The Sprint 3 picker later considered **162,461 preliminary candidates**, screened for distinct combinations of `SKILL.md` content and sibling-file state, and required at least one usable directed comparison. These intermediate counts describe different screening stages; **none is the denominator for the reported 21.0% result**. The final study contains 300 relationship-eligible families and 6,570 loaded occurrences.

Local artifact and group IDs may differ across database builds; reproduction uses repository/path identity, content hashes, and the dataset snapshot. Sibling enrichment distinguishes known empty bundles from missing content.

Sprint 3 Sample Audit

The following counts come from the saved `sampling_report.json` files. A rejection means that the candidate failed the family picker's relationship validation, **not** that it contained no security-sensitive behavior. The picker replaced rejected candidates in the same stratum.

Each trial retains the same 20/55/20/4/1 size allocation described in Section IV. The sampling reports preserve the rejection reasons, source database identity, thresholds, excluded `families.txt` paths and checksums, and exact selected names. The screened eligible population was not enumerated for population weighting, so these records cannot support a corpus-wide prevalence estimate.

Appendix B Pipeline Implementation

Detection Rule Registry

The saved Sprint 3 manifests identify **15 active rules across six categories**. These rules recognize candidate evidence, while the eleven risk indicators (RIs) in Section V assess whether *changes* in that evidence satisfy specific criteria. A new rule match, increased rule count, or changed command text alone does not establish an RI.

The executable definitions are in `src/gitskills/rules/registry.py` and the category modules. For example, `EXT-002` covers explicit package installation, `EXT-004` covers manifest-based dependency installation, and `EXT-003` detects staged downloaded-file execution. Overlapping rules can describe the same command, so individual matches are not independent events. Leading `SKILL.md` frontmatter is masked for behavioral analysis; sibling-file content is scanned separately when eligible and available. Dataset instructions are never executed.

End-to-End Workflow and Saved Policy

For each selected family, the pipeline loads all qualifying artifacts, forms content variants and similarity clusters, evaluates the direction hierarchy, selects primary predecessors, and scans the directed pairs. It extracts evidence from both skill bodies and eligible added or changed sibling files, compares the base and derived evidence, applies RI-001 through RI-011, and records uncertainty and missing content. Cached family scans are reused only when their source, implementation, rule, and policy fingerprints are compatible.

Direction priority is declared source, direct chronology, equivalent-peer chronology, repository-creation boundary, then asymmetric containment. These are inference rules, not proofs of copying. Additional directed edges exist in the analysis method but **were not scanned for the three Sprint 3 result sets**. Ambiguous directions and equivalent peers remain available for relationship review without being treated as evidence that a behavior was introduced.

Skill and Sibling-File Change Classification

Similarity equivalence does not require the same byte hash, so a `siblings-only` comparison can still contain textual differences in `SKILL.md`. Added or changed sibling **files** with available nonempty content are eligible for static scanning, regardless of extension. Unavailable content is reported as missing; paths, filenames, and directory changes alone are not treated as the behavior of an unknown script.

Relationship and Scanning Diagrams

The first diagram shows how the analyzer separates relatedness, direction, equivalent peers, and selected versus additional edges. It is a **conceptual diagram**; only selected primary edges enter the Sprint 3 security counts. The second diagram outlines the family-scanning component, including conditional sibling analysis. Risk-indicator assessment and cohort aggregation follow the underlying evidence comparison.

Preparation stage	Retained artifacts	Excluded at this stage
All source occurrences	3,797,117	-
Valid frontmatter	1,627,753	2,169,364
Body over 50 characters and required fields present	1,621,757	5,996
Name repeated among qualifying occurrences	943,257	678,500

Table 8: Candidate-selection audit for the prepared GitSkills snapshot

Sample	Seed	Families selected	Candidates rejected during drawing	Previously selected families excluded
Trial 1	5801	100	239	0
Trial 2	5802	100	288	100
Trial 3	5803	100	247	200

Table 9: Sprint 3 sample seeds and family-selection audit

Category	Active rules	What they recognize	Main limitation
Command execution	`CMD-001`, `CMD-002`	Shell-command blocks and recognized command invocations	A code example is not proof of execution.
Network access	`NET-001`, `NET-002`	HTTP(S) URLs and recognized network clients	A documentation link alone is not a network operation.
File-system access	`FS-001`, `FS-002`, `FS-003`	File commands, writing APIs, and reading commands/APIs	Ordinary file use is not automatically sensitive or destructive.
Credentials	`CRED-001`, `CRED-002`	Secret-related references and recognized credential files	A name or placeholder does not prove access to a secret.
External code and dependency acquisition	`EXT-001`, `EXT-002`, `EXT-003`, `EXT-004`	Direct download-to-shell, explicit package installs, staged download-and-execute, and manifest-based dependency installs	A legitimate installation may match; multi-step behavior may be incomplete.
System modification	`SYS-001`, `SYS-002`	Privilege elevation and system-changing commands	Administrative instructions can be legitimate.

Table 10: Fifteen detection rules and their coverage limits

Setting	Sprint 3 value
Main study	Three non-overlapping, 100-family stratified samples
Family seeds	5801, 5802, 5803
Manual review seed	580
Edge selection	`primary` only
Shingle width	5 tokens
Minimum containment	0.80
Minimum Jaccard	0.30
Minimum shared shingles	5
Containment direction margin	0.10
Detection rules / risk indicators	15 / 11
Source code revision	`ae69732ed8988ad7066ff351848f7d2e8ad311a1`

Table 11: Pinned analysis configuration for Sprint 3

Change type	Meaning
`skill-only`	`SKILL.md` content differs; known sibling state is equivalent.
`siblings-only`	`SKILL.md` content is equivalent **under the analyzer's similarity representation**; known sibling state differs.
`skill+siblings`	`SKILL.md` content and known sibling-file content both differ.
`equivalent`	Both `SKILL.md` content and known sibling-file states are equivalent.
`unknown`	Missing or uncertain sibling state prevents reliable bundle classification.

Table 12: Bundle change classifications

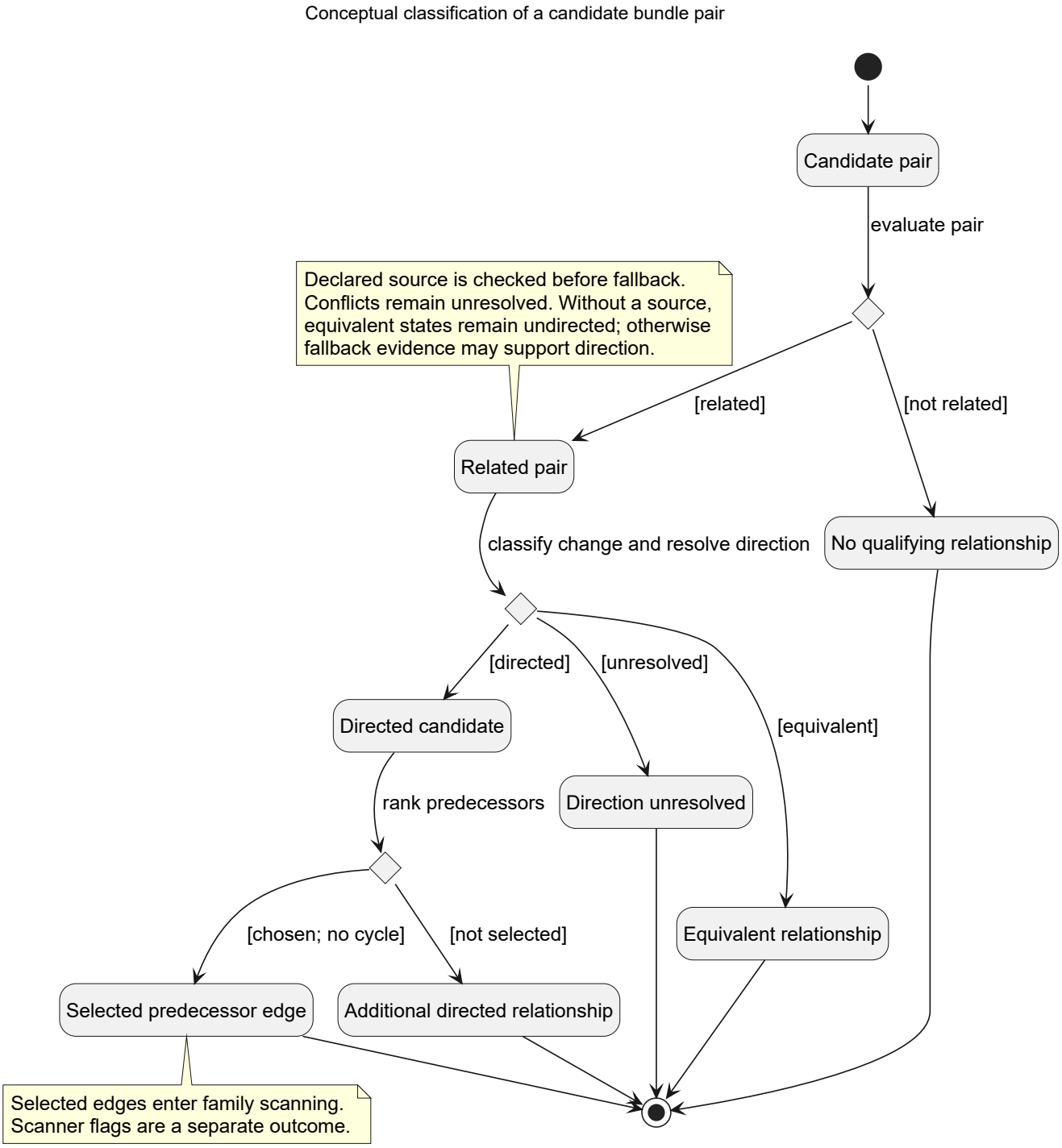


Figure 4: Conceptual UML state diagram for relationship classification. Scanner flags are separate from lineage outcomes.

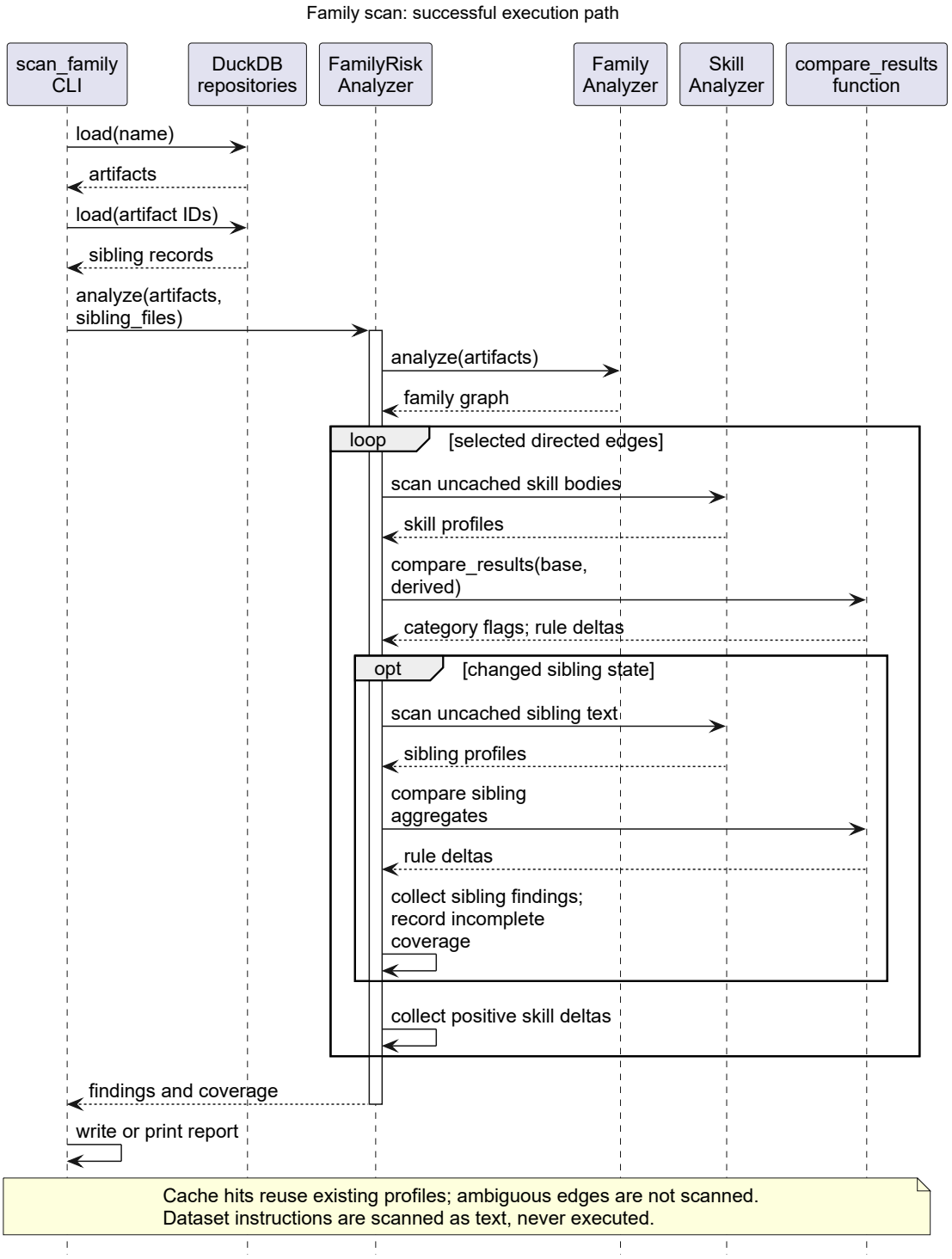


Figure 5: Family scanning and conditional sibling analysis; shown as a component-level summary rather than the complete research aggregation.

Appendix C Validation Scope and Status

Each of the **three Sprint 3 trials** generated **30 directed-comparison review cases** (five from each of six strata) and **15 relationship-review cases** (five from each of three strata), using review seed **580**. The total is **90 directed cases and 45 relationship cases**. These samples come from the saved results for each separate 100-family trial; they are **not** additional data collected for the final analysis.

Directed review covers supported indicators, uncertain indicators, flagged comparisons without supported indicators, coverage-limited comparisons, sibling changes, and unflagged directed comparisons. Relationship review includes equivalent, ambiguous, and no-related-edge cases. The two reviews serve different purposes: relationship review examines the similarity and ordering decisions, while directed review examines extraction, source differences, RI judgments, and coverage.

Reviewers distinguish rule matches, extracted evidence, supported versus uncertain indicator decisions, missing evidence, and evidence for direction. A reported RI is a **false positive** when manual inspection does not support it under the defined indicator policy. A **false negative** requires a manually supported *in-scope* indicator that the scanner did not report as supported. A reviewer should not count a benign change outside RI-001 through RI-011 as a missed indicator merely because it is security-related in another sense.

At the time of the saved runs, the annotation templates and selected cases existed, but **no independently adjudicated Sprint 3 validation metrics were included** in the summaries. Manual review is ongoing. We therefore do not report precision, recall, reviewer agreement, or confirmed positive/negative counts yet. When adjudicated files are available, the pipeline can recompute the validation portion of each saved summary without changing the original sample or scanner outputs.

The stratified review samples help characterize error types and coverage gaps, but their raw proportions should not be treated as population-wide accuracy estimates without a justified sampling calculation. Detailed review instructions are in `docs/instructions/manual_annotation_protocol.md`; the saved case lists and manifest checksums are linked in Appendix E.

Appendix D Additional Sprint 3 Results

The tables below are calculated from the saved Sprint 3 `review/summary.json`, `review/families.csv`, `review/directed_pairs.csv`, and `review/direction_basis.csv` files. All indicator results are **automated** and pending independent human adjudication. Per-indicator counts overlap: one comparison may contain more than one supported indicator, and multiple comparisons may share artifacts.

Indicator Frequency by Trial

Table 13 counts primary directed comparisons supporting each risk indicator across the three trials. Its final column shows how many distinct families had at least one such comparison.

Direction Evidence and Selected Comparisons

Table 14 groups the 2,157 primary directed comparisons by the evidence used to infer their order. Each comparison belongs to one direction category. These are inferred comparisons, not verified copying histories.

Bundle Change Types

Table 15 groups the primary comparisons by changes to `SKILL.md` content and known supporting files.

Change type	Comparisons	With supported RIs
`skill-only`	805	33
`siblings-only`	190	41
`skill+siblings`	804	242
`unknown`	358	24
Total	**2,157**	**340**

Table 15: Primary-comparison counts by bundle change type

`unknown` describes an unavailable or insufficiently known sibling state, not a known unchanged bundle. Equivalent relationships are not counted as base-to-derived comparisons. A `siblings-only` classification uses equivalence of `SKILL.md` content under the similarity representation rather than requiring identical byte hashes; see Appendix B.

Family-Size and Cluster Notes

Across 300 families, the analyzer produced **438 similarity clusters**. Families with multiple clusters were uncommon in the smaller strata (0 of 60 Low families; 10 of 165 Medium) and more common in the larger strata (19 of 60 Large; all 12 X-Large; all three Extreme). These clusters indicate relatedness under the project's token-shingle method, not verified independent origins.

Table 16 illustrates how artifact counts and similarity-cluster counts differ among four sampled families.

Family	Artifact occurrences	Similarity clusters
`implement`	856	35
`mcp-builder`	543	11
`documentation`	508	19
`cso`	150	2

Table 16: Examples of family size and similarity clusters

The full per-family counts, coverage records, and selected comparisons remain in the three saved `review/` directories. They are not duplicated here because the detailed tables are best inspected directly as reproducible data files.

Indicator	Trial 1	Trial 2	Trial 3	All comparisons	Distinct families
RI-001	0	0	3	3	3
RI-002	0	0	0	0	0
RI-003	8	3	41	52	17
RI-004	0	0	0	0	0
RI-005	25	15	115	155	26
RI-006	57	7	95	159	10
RI-007	3	0	1	4	2
RI-008	8	4	6	18	8
RI-009	0	0	0	0	0
RI-010	33	23	145	201	39
RI-011	0	0	1	1	1

Table 13: Risk-indicator frequency across the three trials

Direction basis	All comparisons	With supported RIs	Incomplete evidence coverage
Declared source	10	0	1
Direct chronology	399	55	42
Equivalent-peer chronology	355	43	8
Repository-creation boundary	827	111	86
Directional containment	566	131	74
Total	**2,157**	**340**	**211**

Table 14: Direction evidence and coverage for selected comparisons

Appendix E Supporting Repository Material

The report uses the **three saved Sprint 3 trials**, not the earlier Sprint 2 exploratory cohort. The source code revision recorded in all three manifests is `ae69732ed8988ad7066ff351848f7d2e8ad311a1`; the manifests additionally retain the implementation fingerprint, active rules, dataset identity, and selected inputs. Results are not recomputed by rebuilding the report.

The saved research inputs and outputs are organized as follows (each trial has the same layout):

```
notebooks/end_to_end_pipeline/sprint3/
  samples1/families.txt
  samples1/sampling_report.json
  samples2/families.txt
  samples2/sampling_report.json
  samples3/families.txt
  samples3/sampling_report.json
  results1/
    sample_manifest.json
    execution_summary.json
    review/summary.json
    review/families.csv
    review/directed_pairs.csv
    review/indicator_frequency.csv
    review/direction_basis.csv
    manual-review/directed-pairs/
    manual-review/relationship-pairs/
  results2/                  # same structure
  results3/                  # same structure
```

The `sampling_report.json` files record sampling seeds **5801–5803**, stratum allocations, exclusions, and candidate rejections. The `sample_manifest.json` files record the selected families, policies, scanner and code identities, review sample checksums, and a source fingerprint. Research summaries and CSV files contain the automated outcome counts used in Sections IV–VI and Appendix D; audit and execution files provide coverage and processing status. The separate annotation files are reviewer-owned and remain distinct from automated predictions.

Two cross-trial charts in Section VI are generated from the saved `families.csv` and `directed_pairs.csv` files using `report/figures/generate_sprint3_figures.py`. The script does not alter trial results or select new families.

The three sample manifests currently mark their approval reference as **provisional** (`sample_status: provisional`, no approval reference). The final report should not imply formal approval has been recorded unless the manifest or repository decision history is updated. A tagged release and final manual-validation record should be linked when available.

Appendix F Use of AI in Development and Report Preparation

Project Group 8 used AI assistance during tool and pipeline development and in preparing this report. Development assistance included discussing implementation approaches, drafting and revising code, troubleshooting errors, and supporting documentation and verification work. These activities contributed to the components integrated into the end-to-end pipeline.

For the report, AI assistance supported organizing material, drafting and revising prose, condensing earlier documentation, and checking consistency between the narrative and saved outputs. It also supported formatting and rebuilding the report. The numerical findings come from the saved pipeline artifacts; AI-generated explanations are not experimental measurements.

Repository resource	Purpose
End-to-end notebook	Reproduces cohort processing and result tables.
Pipeline README	Configuration, outputs, cache, and validation instructions.
Dataset preparation	GitSkills acquisition and DuckDB preparation.
Candidate-selection SQL	Generates the comparison candidate pool (Appendix A).
Sibling fingerprint SQL	Represents known versus unknown bundled-file content.
Sampling decision RDR-014	Family eligibility and stratified sampling.
Edge-selection decision RDR-013	Rationale for primary-only comparisons and family-level reporting.
Risk indicator definitions RDR-010	Operational scope of RI-001 through RI-011.
Threat model	Trust boundary, detection scope, and security interpretation limits.
Annotation protocol	Independent review and adjudication guidance.
Report build guide	Source sections, figures, and local report rendering.

Table 17: Reproducibility and supporting repository resources

Project Group 8 remains responsible for the implementation, interpretation, and final submitted text. Generated code and prose require checking against project requirements, source evidence, and reproducible outputs. AI assistance does not establish the correctness of inferred relationships or reported indicators, and it does not resolve the validation limitations stated in the main report.